Flight Mechanics, Infinite Object Field, and Rear-View Camera

The ship is a THREE.Group that holds both cameras so moving and rotating the group moves the whole view. Each frame, the mouse position is normalized to the range -1 to 1 and used as a turn rate rather than a fixed angle: yaw and pitch change by mouse offset * turnSpeed * deltaTime, and pitch is clamped so the ship can't flip over. The ship then moves along camera.getWorldDirection() at shipSpeed * deltaTime, so it always flies toward the center of the screen at the same rate on any computer, with W & S changing the speed. The infinite field reuses a fixed pool of 150 asteroids. Any asteroid farther than fieldRadius from the ship is moved to a random point 70-100% of that radius away. A dot product with the forward vector flips the point to the front if it would land behind the ship. The rear view mirror is a second camera, rotated 180 degrees and attached to the ship. It is drawn in a second pass using setViewport and setScissor with the scissor test enabled so only the mirror's rectangle is cleared and redrawn. Because WebGL measures y from the bottom of the screen, the rectangle's y is windowHeight - mirrorHeight - margin. The border is made by clearing a slightly larger scissor rectangle to blue before the mirror is drawn.

Asteroid Geometry, Texturing, Shaders, and Motion

The object geometries are all kept in an array of ObjectGeometries[], which features a variety of random asteroid-ish shapes such as icosahedrons, octahedrons, dodecahedrons, boxes, spheres, and tetrahedrons. There is additionally a colors array that is filled with bright, vibrant colors (easy to tell for blending later). For each asteroid object, it is randomly given a number (shapeID) that corresponds to an object in ObjectGeometries[], and the same for the color of the object. Textures were simply mapped on the object using THREE's textureLoader and then loaded in the asteroid texture as a .jpg link I found on wikipedia, then put that texture in the

object's material. These are all applied in the vertex/fragment shaders, where not only the texture and colors get passed on, but additionally both ambient lighting and directional lighting is applied here. The ambient lighting applies equal lighting to all meteors in the fragment shader, and the directional lighting uses the sun's position to apply lighting based on the faces that do/don't face the sun. Lastly, each object is given a random drift amount which is their linear velocity as they move through space, by updating position based on it. However, they are also given a random number between 0 and 1 called driftType. If their driftType is 0 then they keep their linear drift through space, however if their driftType is 1 then their drift slightly curves by a random amount, and functions on the sin function.

Collision Detection and Collision Reaction

Collision detection uses bounding spheres around the ship and asteroids. When a collision occurs, the asteroid's drift direction is changed so it bounces away from the ship. An *isColliding* Boolean prevents the same collision from triggering repeatedly. A brief screen shake is also applied to the main POV camera during a collision. The camera moves slightly on the X and Y axes for about 0.25 seconds, then returns to its normal position.

Above and Beyond

The Above and Beyond feature is an interactive cockpit HUD in the upper-left corner. It displays the ship's speed, heading, pitch, distance traveled, flight time, FPS, and number of collisions. The information updates during flight to give the user real-time feedback about the ship and the simulation.